

Creating Curriculum Of Unknown Unknowns By Battling Machine Opponents

Anand Sriraman, Shirish Karande

TCS Research - TRDDC

Tata Research Development and Design Centre

54-B, Hadapsar Industrial Estate

Pune, Maharashtra, India - 411013

{anand.sriraman, shirish.karande}@tcs.com

Abstract

In this paper, we seek to understand whether the *Beat the Machine* protocol can be extended to a scenario where a human can choose to play against machines of different capabilities. By playing against these machines, we hope to understand the human's implicit ranking of the unknown unknowns discovered. We then use the examples, the choice of machine opponent, and the confidence of the machine opponent to generate a curriculum, which can be used for better training of image classification systems.

Introduction

Human-in-the-loop systems have been used to train predictive machine learning models. However, predictive models may produce incorrect labels with high confidence (unknown unknown). Solutions proposed to solve this include using feedback from an oracle (Lakkaraju et al. 2017) and using crowds to find the "unknown unknowns" (Attenberg, Ipeirotis, and Provost 2011).

The *Beat the Machine* (BTM) protocol introduced by Attenberg et al., is an interesting method to find the unknown unknowns for machine classifiers. Participants are asked to submit data points, which they think will be able to fool the machine into misclassifying. A reward is given for getting a misclassification, and the quantum of reward depends on the quality of the submission.

Re-training a classifier with the unknown unknowns discovered from the BTM process can lead to improvement in the classifier performance. We believe that this performance can be further enhanced if the unknown unknowns are presented as a curriculum for the machine. Curriculum Learning has been proposed as a method to speed up machine learning by providing training data in a meaningful order, to gradually teach more complex concepts and speed up training (Bengio et al. 2009).

We propose extending the BTM protocol to allow participants to compete against multiple versions of the classifier. By competing against multiple versions of a classifier, we may find cases where examples perceived to be less complex by humans are more complex for the classifier to learn. Using the choices made by the participants and the outputs of

the classifier, we propose a method to generate the curriculum consisting of examples submitted by the participants.

Experiment Design

To simulate machines of differing capabilities, we trained an image classification system in an iterative setting, increasing the size of the training set on each iteration. The classification pipeline consisted of processing the image through a pre-trained ResNet neural network trained on the ImageNet dataset (He et al. 2016). We extracted the output from the average pooling layer for each image. We use this output as a feature representation for the image to train a classifier. We stored each version of the machine thus trained. The training set consisted of images from five classes of common household appliances: Air Conditioners, Washing Machines, Laptops, Televisions and Refrigerators.

Figure 1 shows the user interface shown to participants. They're asked to choose the version of the classifier they'd like to compete against along with their image and the true class of the image. They are shown 5 versions of the machine to choose from, each coming from a later iteration than the previous.

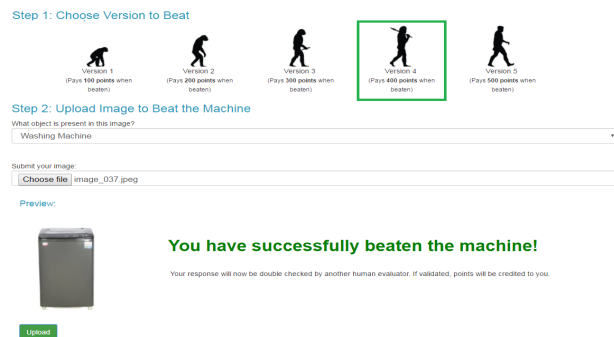


Figure 1: Submission User Interface

When users submit their image, their submission is evaluated by all trained models currently available. The trained models are hosted as a web service using the Flask web framework for Python. These web services are called to get each model's prediction and show it to the user. If the prediction from their chosen version does not match the true

class they provided, they are told that they have *beaten the machine*.

Successful submissions are then sent to crowd workers, recruited from CrowdFlower, to verify that their provided class actually matches their image. If verified, points are credited to the user. For verification, we get multiple crowd workers to verify the image with a binary response. These responses can be aggregated by using an EM-based consensus algorithm (Welinder and Perona 2010) or a Belief Propagation inspired approach described in (Karger, Oh, and Shah 2011).

We only show workers information about whether they defeated their chosen machine opponent. We do not show them the outputs from the other machine classifiers as we do not want to bias their future choices. We want to understand their perceptions of complexity of the images, which we use to create the curriculum.

Incentivizing Participation

Incentivizing Submissions We want to attract participation from users on social media such as Facebook, Twitter and our organisations’s internal social network. Hence, the reward and participation mechanism is very important. We want to focus on two aspects of incentivisation : a) incentivizing participation to attract users, and b) incentivizing choosing the appropriate version of the machine to battle against.

For the *first aspect*, each machine rewards the user with some reward points when beaten. Higher versions of the machine give higher reward. There’s also a leaderboard which displays the top 20 participants overall based on the points earned along with the rank of the user. The leaderboard page can be shared via social media, hopefully spurring participation. We’ve also tuned the website to be responsive on mobile devices. This will allow users submit images using the camera on their smartphones, making it easier to participate.

For the *second aspect*, we want to ensure that the users are carefully choosing which version of the machine they want to battle against. Ideally, we want them to choose the highest version of the machine that they believe their submission can defeat. Hence, easier versions from earlier iterations reward lesser than machines from later iterations. In our implementation, the lowest version pays out 100 points when defeated, going all the way up to 500 points for the highest version. However, this isn’t enough as users may just choose random versions to play against. Hence, we introduced a cost of 25 points to make a submission. Since machines only pay out when they’re defeated, users are made to choose carefully about the choice of machine. We also debated having higher risk-reward ratios for each machine. We decided against this mechanism however, as users with different risk appetites may choose get biased, and this may not reveal a perceived curriculum.

Incentivizing Verification Verification is an important step in our system since points aren’t awarded to users till their submission has been validated by crowd workers. We also need submissions to be validated as quickly as possible, since updates to the leaderboard will incentivize participa-

tion. Our system bundles a batch of 50 submissions to create a Job on CrowdFlower using their API. Each submission is validated by 5 workers and each worker validates 10 submissions. We pay 5 cents to each worker for their 10 validations.

Generating the Curriculum

The curriculum is essentially a ranking of the dataset of submissions ordered by increasing levels of complexity. To test whether a curriculum of unknown unknowns is useful, we trained a 3 layer neural network on the shapes data mentioned in (Bengio et al. 2009). We trained the network on a small subset of the training data, arranged in a curriculum. This network was then used to obtain unknown unknowns from the remaining training data. We found that when re-training the neural network, interlacing the unknown unknowns with the original training data while preserving the curriculum was performing better than when the unknown unknowns were added randomly.

To obtain the curriculum, we generate pairwise comparisons of the data based on the choices made by the user and the posteriors obtained from the classifier. Aggregating these pairwise comparisons to form a final ranking can be performed by methods available in literature (Chen et al. 2013), (Wauthier, Jordan, and Jojic 2013).

When workers choose to battle against a particular version, they are providing an implicit ranking for that submission. That example is more complex than all submissions played against lower versions, and less complex than all submissions played against higher versions. Using this, we generate the user’s pairwise ranking for his/her submission.

Each submission is also evaluated by all the versions of the machine classifier. The classifier gives us a classification confidence on each submission. To obtain the classifier’s pairwise rank interpretations, we rank the submissions based on the confidence score obtained. Let c be the confidence score for the correct class in the posterior. For *known known* submissions, where the classifier got it right, we use c to rank. For *unknown unknown* submissions, where the classifier got it wrong we use $(1 - c)$ to rank.

By using both human and machine’s perception of complexity, we should be able to take care of some interesting cases. There may be some examples which the human deems of quite low complexity and chooses a low version to battle against. However, we may find that even the more advanced machines are unable to classify it correctly. At the other end of the spectrum, we may find that humans overestimate the complexity of an example by choosing to battle against a higher version. These examples may be easy to classify for the machine. Hence using the perception of both parties, we hope to arrive at a better curriculum.

Future Work

This paper discusses the design of a system to creating curriculums to train machine learning algorithms using human and machine perceptions of complexity. This system is still a work-in-progress. We hope to launch the experiment soon and discover the effectiveness of the curriculum thus produced.

References

- Attenberg, J.; Ipeirotis, P. G.; and Provost, F. J. 2011. Beat the machine: Challenging workers to find the unknown unknowns. *Human Computation* 11(11).
- Bengio, Y.; Louradour, J.; Collobert, R.; and Weston, J. 2009. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, 41–48. ACM.
- Chen, X.; Bennett, P. N.; Collins-Thompson, K.; and Horvitz, E. 2013. Pairwise ranking aggregation in a crowd-sourced setting. In *Proceedings of the sixth ACM international conference on Web search and data mining*, 193–202. ACM.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.
- Karger, D. R.; Oh, S.; and Shah, D. 2011. Iterative learning for reliable crowdsourcing systems. In *Advances in neural information processing systems*, 1953–1961.
- Lakkaraju, H.; Kamar, E.; Caruana, R.; and Horvitz, E. 2017. Identifying unknown unknowns in the open world: Representations and policies for guided exploration.
- Wauthier, F.; Jordan, M.; and Jojic, N. 2013. Efficient ranking from pairwise comparisons. In *International Conference on Machine Learning*, 109–117.
- Welinder, P., and Perona, P. 2010. Online crowdsourcing: rating annotators and obtaining cost-effective labels.